

ASSIGNMENT 2

INDICE

INDICE.....	1
VERY BUSY EXPRESSIONS.....	2
DESCRIZIONE DEL PROBLEMA.....	2
DOMINIO.....	2
DIRECTION.....	2
FUNZIONE DI TRASFERIMENTO.....	3
MEET OPERATOR.....	3
BOUNDARY CONDITIONS.....	3
INITIAL INTERIOR POINT CONDITIONS.....	3
ANALISI DELL'ESEMPIO.....	5
DOMINATOR ANALYSIS.....	6
DESCRIZIONE DEL PROBLEMA.....	6
DOMINIO.....	6
DIRECTION.....	6
FUNZIONE DI TRASFERIMENTO.....	7
MEET OPERATOR.....	7
BOUNDARY CONDITIONS.....	7
INITIAL INTERIOR POINT CONDITIONS.....	7
ANALISI DELL'ESEMPIO.....	8
ANALISI DEL PROBLEMA.....	9
CONSTANT PROPAGATION.....	9
DESCRIZIONE DEL PROBLEMA.....	9
DOMINIO.....	10
DIRECTION.....	10
FUNZIONE DI TRASFERIMENTO.....	10
MEET OPERATOR.....	10
BOUNDARY CONDITIONS.....	11
INITIAL INTERIOR CONDITIONS.....	11
ESEMPIO.....	12
ANALISI DELL'ESEMPIO.....	14

VERY BUSY EXPRESSIONS

DESCRIZIONE DEL PROBLEMA

Dato uno specifico punto P all'interno del CFG, una expression E viene definita come Very Busy expression rispetto al punto P se viene valutata in ogni path tra il punto P e l'EXIT BLOCK prima che uno dei suoi operandi venga ridefinito.

Affinché un'espressione sia considerata Very Busy in un punto P del CFG, devono essere soddisfatte le seguenti due condizioni:

1. L'espressione deve essere valutata lungo ogni possibile cammino (path) che origina dal punto P e conduce all'EXIT BLOCK.
2. Non deve verificarsi alcuna ridefinizione (modifica) degli operandi che compongono l'espressione nel path compreso tra il punto P e il punto di valutazione dell'espressione analizzata.

DOMINIO

Il dominio è l'insieme delle espressioni che vengono valutate all'interno di ciascun Basic Block del CFG, dove con espressione si intende specificatamente il right hand side value.

E' possibile rappresentare questo dominio attraverso l'utilizzo di un bit vector, dove ciascuna posizione è il riferimento ad una specifica espressione del CFG e rappresenta la "presenza" o "assenza" della proprietà di Very Busy expression in corrispondenza di uno specifico punto del CFG.

In questo modo, il bit vector è in grado di mostrare, per ciascun punto del grafo, quali espressioni sono considerate come "Very Busy" in quella specifica posizione.

DIRECTION

Riprendendo la definizione di Very Busy expression: per poter definire se una espressione E è una Very Busy expression in un punto P è necessario analizzare la validità di tale proprietà in ciascuno dei path che partono dal punto P considerato fino al EXIT block del CFG.

Questo evidenzia come la natura Very Busy è definita sulla base dell'analisi delle istruzioni successive al punto P, portando così la verifica del basic block in maniera Backward, ovvero ricevendo le informazioni al punto di OUT, calcolare la validità della proprietà attraverso la funzione di trasferimento e fornire i risultati in corrispondenza del punto IN del basic Block.

FUNZIONE DI TRASFERIMENTO

$$f_B(X) = GEN[B] \cup (OUT[B] - KILL[B])$$

$$OUT[B] = \bigwedge IN[Si] \forall Si \text{ Successore di } B$$

$$IN[B] = f_b(OUT[B])$$

Dove specificatamente:

- GEN[B] - Gen set: insieme delle espressioni che sono state valutate all'interno del basic block B considerato e che non sono precedute da una ridefinizione di uno dei suoi operandi.
- KILL[B] - Kill set: È l'insieme di tutte le espressioni che utilizzano almeno un operando che è stato ridefinito da una espressione all'interno del basic block B considerato.

MEET OPERATOR

Riprendendo la definizione di very busy expression, per fare in modo che in un punto P una expression E sia considerata come "very busy" è specificatamente necessario che lo sia in ciascuno dei suoi path che vanno dal punto P fino al punto exit.

Il vincolo che lo debba essere in tutti i suoi percorsi suggerisce che il meet operator sia l'intersezione

In questo modo una espressione è considerata come very busy nel punto di ingresso del basic block (OUT essendo in backward direction) solo nel caso in cui lo sia in ciascuno dei suoi successori.

BOUNDARY CONDITIONS

il punto IN dell'EXIT BLOCK è inizialmente vuoto, dovuto al fatto che nessuna espressione è considerata Very busy nel punto di EXIT, poiché non è né valutata e non ha nemmeno successori.

INITIAL INTERIOR POINT CONDITIONS

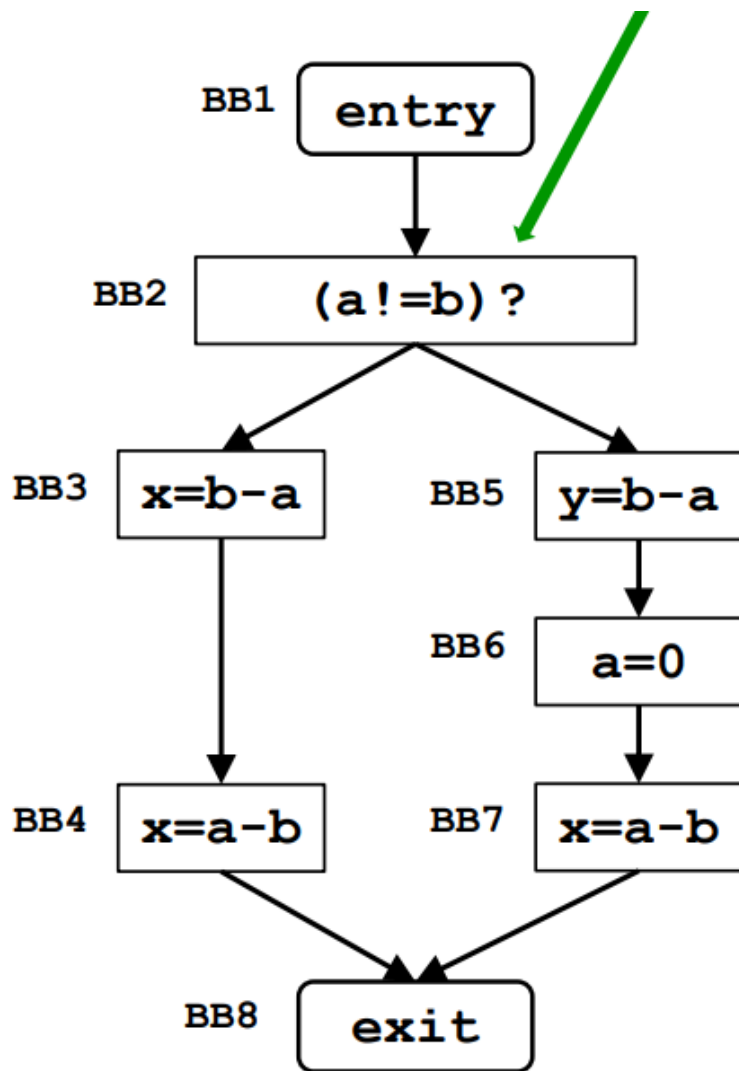
Essendo il meet operator l'intersezione, per fare in modo che in un punto di intersezione le condizioni iniziali di un successore non ancora valutato influiscano sul risultato delle funzioni di trasferimento di un altro successore valutato, si utilizza l'insieme universale.

Infatti l'intersezione fra un insieme OUT[Bi] e l'insieme universale dà come risultato l'insieme OUT[Bi] stesso, non influenzando così il risultato della funzione di trasferimento del predecessore analizzato.

	Dataflow Problem: Very Busy expression
--	--

Domain	Espressioni
Direction	backward
Transfer function	$f_B(X) = GEN[B] \cup (OUT[B] - KILL[B])$ $OUT[B] = ^{IN}[Si] \quad \forall Si \text{ Successore di } B$ $IN[B] = f_b(OUT[B])$
Meet operation (^)	$\cap$ intersezione
Boundary conditions	$\emptyset$ insieme vuoto
Initial interior point	U Insieme universale

ANALISI DELL'ESEMPIO



BIT VECTOR: < a!=b, b-a, a-b >

	ROUND 1		ROUND 2		ROUND 3	
	IN	OUT	IN	OUT	IN	OUT
BB1	/	<100>	/	<110>	/	<110>
BB2	<110>	<010>	<110>	<010>	<110>	<010>
BB3	<011>	<001>	<011>	<010>	<011>	<010>
BB4	<001>	<000>	<010>	<000>	<010>	<000>
BB5	<010>	<000>	<010>	<000>	<010>	<000>
BB6	<000>	<001>	<000>	<001>	<000>	<001>

BB7	<001>	<000>	<001>	<000>	<001>	<000>
BB8	/	<000>	/	<000>	/	<000>

DOMINATOR ANALYSIS

DESCRIZIONE DEL PROBLEMA

In un CFG, un nodo X domina un altro nodo Y se il nodo X appare in ogni percorso del grafo che porta dal blocco ENTRY al blocco Y

- Annotiamo ogni basic block B_i con un insieme $DOM[B_i]$.
 B_i appartiene $DOM[B_j]$ se e solo se B_i domina B_j
- Per definizione un nodo domina sé stesso: B_i appartiene a $DOM[B_i]$

DOMINIO

Il dominio del problema è composto dall'insieme di tutti i basic block presenti nel CFG, poiché la Dominator analysis lavora esplicitamente a livello di intero blocco. Di conseguenza, l'analisi può essere implementata attraverso un array di bit associato a ciascun punto del basic block, che definisce, per ogni posizione, quali sono i blocchi dominatori.

DIRECTION

Riprendendo la definizione di Dominanza, per determinare se uno specifico blocco Y viene dominato da un altro blocco X è necessario verificare se tale blocco X compare in ciascun percorso che dall ENTRY block porta all'ingresso del blocco Y.

Di conseguenza per poter analizzare i dominatori di un basic block preso in esame, è necessario dover osservare il CFG partendo dai suoi predecessori, suggerendo così una analisi in direzione Forward.

La conoscenza della proprietà di dominanza entra in corrispondenza del punto IN di ogni basic block, viene elaborata dalla funzione di trasferimento e viene fornita ai suoi successori dal punto di OUT.

FUNZIONE DI TRASFERIMENTO

$$f_B(X) = GEN[B] \cup OUT[B]$$

$$IN[B] = \bigwedge OUT[Si] \quad \forall Si \text{ Predecessore di } B$$

$$OUT[B] = f_b(IN[B])$$

Dove:

- GEN[B] - gen set: è il basic block stesso che viene analizzato, poiché ogni basic block è dominatore di se stesso per definizione.
- kill set: non viene considerato, non essendoci alcuna operazione che “rimuove” la proprietà di dominanza

MEET OPERATOR

Riprendendo la definizione di dominanza, per fare in modo che un basic block Y sia dominato da un altro basic block X è necessario che X compaia in tutti i percorsi tra il blocco di ENTRY ed Y, suggerendo così di usare l'intersezione come meet operator. Attraverso l'intersezione, in corrispondenza di un punto di unione di più path, si considerano come dominatori del blocco Y preso in esame solo i basic block che sono dominatori di tutti i suoi predecessori.

BOUNDARY CONDITIONS

All'inizio della analisi nessun basic block è ancora stato analizzato e, di conseguenza, nessuna relazione di dominanza è stata definita.

Questo suggerisce che l'insieme di partenza è l'insieme vuoto.

Nel caso in cui però l'ENTRY block venga considerato come blocco effettivo del CFG, allora l'insieme di partenza è composto solamente dall'ENTRY block stesso.

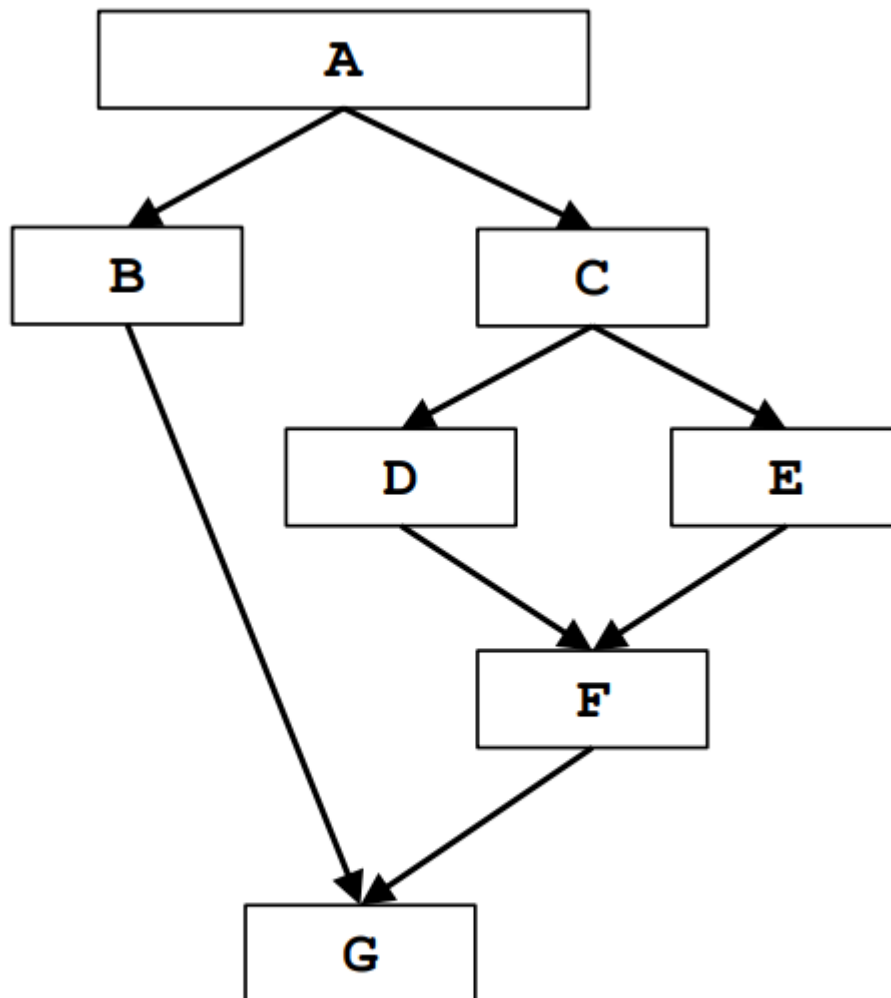
Questo grazie alla definizione di dominanza dove ogni blocco è dominatore di se stesso.

INITIAL INTERIOR POINT CONDITIONS

Essendo il meet operator l'intersezione, le initial interior point conditions per ogni Basic block sono l'insieme universale U.

Questa scelta è stata fatta per evitare che i risultati ottenuti dalle analisi dei basic block già valutati siano influenzati da un basic block non ancora valutato, che potrebbe (nel caso in cui fosse considerato come insieme vuoto) annullare tutte le proprietà di dominanza ottenute. Infatti l'intersezione fra il punto di OUT di un basic block analizzato (che contiene i suoi dominatori) e l'insieme universale dà come risultato l'insieme OUT stesso, non influenzando così il risultato.

ANALISI DELL'ESEMPIO



$$\text{DOM}[F] = \{A, C, F\}$$

	Dataflow Problem: Dominator Analysis
Domain	Basic Blocks
Direction	forward
Transfer function	$f_B(X) = \text{GEN}[B] \cup \text{OUT}[B]$ $\text{IN}[B] = \bigwedge \text{OUT}[Si] \quad \forall Si \text{ Predecessore di } B$

	$OUT[B] = f_b(IN[B])$ $GEN[B] = DOM[B]$
Meet operation (^)	$\cap$ intersezione
Boundary conditions	$\emptyset$ insieme vuoto
Initial interior point	U insieme universale

ANALISI DEL PROBLEMA

	ROUND 1		ROUND 2		ROUND 3	
DOM[BB]	IN	OUT	IN	OUT	IN	OUT
A	/	{A}	/	{A}	/	{A}
B	{A}	{A,B}	{A}	{A,B}	{A}	{A,B}
C	{A}	{A,C}	{A}	{A,C}	{A}	{A,C}
D	{A,C}	{A,C,D}	{A,C}	{A,C,D}	{A,C}	{A,C,D}
E	{A,C}	{A,C,E}	{A,C}	{A,C,E}	{A,C}	{A,C,E}
F	{A,C}	{A,C,F}	{A,C}	{A,C,F}	{A,C}	{A,C,F}
G	{A}	{A,G}	{A}	{A,G}	{A}	{A,G}

CONSTANT PROPAGATION

DESCRIZIONE DEL PROBLEMA

L'obiettivo della constant propagation è quello di determinare in quali punti del programma le variabili hanno un valore costante.

L'informazione da calcolare per ogni nodo N del CFG è un insieme di coppie del tipo <variabile, valore costante>.

Se si ha la coppia <X, C> al nodo N, significa che ad X è garantito avere il valore C ogni volta che N viene raggiunto durante l'esecuzione del programma.

DOMINIO

Il dominio è l'insieme di tutte le coppie < variabile , valore > che si possono valutare nel programma.

Specificatamente si vuole considerare quando una variabile assume sicuramente un preciso valore costante in corrispondenza di uno specifico punto del programma; situazione rappresentata dall'esistenza della coppia <variabile, valore> in corrispondenza di tale punto del CFG.

DIRECTION

Per poter verificare e garantire che ad una variabile X sia sempre associato un valore costante C in uno specifico punto P del CFG, è necessario verificare che esista la coppia <X,C> per ciascun path che dal blocco di ENTRY raggiunga il punto P analizzato. Questa proprietà dipende strettamente dall'analisi dei suoi predecessori, suggerendo così un approccio di tipo Forward.

All'ingresso del basic block IN entrano le variabili fino a quel momento garantite costanti, mentre dal punto di uscita OUT escono le variabili garantite costanti a seguito dell'analisi del basic block (applicazione della funzione di trasferimento).

FUNZIONE DI TRASFERIMENTO

$$f_B(X) = GEN[B] \cup (IN[B] - KILL[B])$$

$$IN[B] = \bigcup OUT[Si] \quad \forall Si \text{ Predecessore di } B$$

$$OUT[B] = f_b(IN[B])$$

Dove:

- GEN[B] - Gen set: rappresenta l'insieme di tutte le coppie <variabile, valore costante> che vengono generate in corrispondenza di tale basic block. Una coppia <X,C> viene generata da un Basic Block nel caso in cui esista una definizione della variabile X di cui si riesca a calcolare il valore costante ad esso assegnato e se tale definizione non sia seguita da una ridefinizione della variabile X stessa.
- KILL[B] - Kill set: rappresenta l'insieme di tutte le coppie <variabile, valore costante> tale per cui la variabile di riferimento viene ridefinita all'interno del basic block.

MEET OPERATOR

Seguendo la definizione di constant propagation, per fare in modo che l'assegnazione di un valore costante C ad una variabile X sia garantito in corrispondenza di un punto P del CFG, è necessario che in tutti quanti gli insiemi forniti dai punti di OUT di ciascuno dei predecessori al punto P sia presente la coppia <X,C>.

Ovvero che la coppia <X,C> sia garantita in ciascuno dei path precedenti al basic block preso in esame.

Questo suggerisce l'utilizzo dell'intersezione come meet operator, dove la coppia $\langle X, C \rangle$ è valida anche nel punto P solo se è presente in ciascuno degli insiemi dei suoi predecessori.

BOUNDARY CONDITIONS

Il punto di inizio dell'analisi è l'ENTRY BLOCK.

Per tale blocco, l'insieme di uscita è inizializzato all'insieme vuoto, poiché, non è ancora stata definita o garantita alcuna assegnazione di valori costanti alle variabili.

INITIAL INTERIOR CONDITIONS

Per evitare che il risultato dell'analisi di un predecessore venga compromesso dall'operazione di intersezione, inizialmente ogni insieme OUT di ciascun basic block viene inizializzato con l'insieme di tutte le possibili coppie $\langle \text{variabile}, \text{valore costante} \rangle$.

	Dataflow Problem: Constant propagation
Domain	Associazione variabile-valore costante $\langle \text{variabile}, \text{valore costante} \rangle$
Direction	forward
Transfer function	$f_B(X) = GEN[B] \cup (IN[B] - KILL[B])$ $IN[B] = \bigwedge OUT[Si] \forall Si \text{ Predecessore di } B$ $OUT[B] = f_b(IN[B])$
Meet operation ($\wedge$)	$\cap$ intersezione
Boundary conditions	$\emptyset$ insieme vuoto
Initial interior point	Insieme di tutte le possibili coppie $\langle \text{Variabile}, \text{Costante} \rangle$

Il problema della constant propagation applicato al Data Flow analysis è principalmente dovuto ai valori iniziali definiti dall'initial interior point conditions in corrispondenza di ciascun punto di OUT di ogni basic block.

Infatti, essendo il meet operator l'intersezione delle coppie variabile-costante $\langle x, c \rangle$ valide nei predecessori di un basic block, se l'OUT di un blocco non è ancora stato definito il suo insieme non deve poter influenzare l'operazione di intersezione con i risultati degli altri predecessori.

Per fare ciò è necessaria la presenza di un insieme che racchiuda tutti i possibili valori costanti assumibili dalle variabili del programma, il quale è possibilmente infinito.

Per questo motivo è possibile affiancare l'insieme delle coppie variabili-valore con un array di stati che ha un riferimento per ogni variabile utilizzata nel programma.

Il vettore di stati viene così valutato in corrispondenza di ogni punto del CFG, definendo quali variabili in quello specifico punto hanno un valore costante garantito e quali invece no.

Dato uno specifico punto P del CFG, l'array di stati assumerebbe i seguenti valori:

- $1 \rightarrow$ nel caso in cui la rispettiva variabile abbia un valore costante garantito nel punto P.
Ovvero che nel punto P, per la variabile x esiste una coppia $\langle x, c \rangle$ nell'insieme dei valori costanti.
- $0 \rightarrow$ nel caso in cui la corrispettiva variabile non abbia un valore costante garantito nel punto P.
Questo potrebbe essere dovuto alla mancanza di una coppia $\langle x, c \rangle$ o della presenza, in un punto di intersezione, di più coppie $\langle x, c \rangle$ ma con valori costanti differenti.
- $-1 \rightarrow$ nel caso in cui una variabile non sia ancora mai stata valutata, e quindi ha uno stato indefinito.
Questo stato definisce le condizioni iniziali in corrispondenza di ciascun punto di OUT di ogni basic block del CFG.

L'introduzione dello stato -1 permette di gestire correttamente l'intersezione tra i predecessori: le variabili in tale stato non vengono valutate nella operazione di meet. In questo modo, il valore finale viene determinato esclusivamente dai predecessori che possiedono una costante definita, evitando che uno stato 'non ancora valutato' comprometta l'analisi.

ESEMPIO

Supponiamo di aver precedentemente valutato l'espressione $x = 5$ nel BB1 (basic block 1). Il risultato sarebbe il seguente:

$OUT[BB1] = \{ \langle X, 5 \rangle \}$

$STATI[BB1] = [X: 1]$ (array di stati dove lo stato di $X = 1$)

Supponiamo che, a seguito del BB1, venga valutato un secondo BB2 che ha due archi entranti: il primo che proviene da BB1 e il secondo che rappresenta un ciclo da un generico basic block BBn.

La conoscenza in ingresso del BB2 sarebbe la seguente:

da BB1:

$OUT[BB1] = \{ \langle X, 5 \rangle \}$

$STATI[BB1] = [X: 1]$

da BBn:

$OUT[BBn] = \{ \text{tutte le possibili coppie } \langle X, C \rangle \}$

$$STATI [BBn] = [X: -1] \rightarrow X \text{ non è ancora stata definita}$$

Essendo -1 lo stato della variabile X in corrispondenza di BBn, tale variabile non viene presa in considerazione per l'operazione di meet.

E quindi:

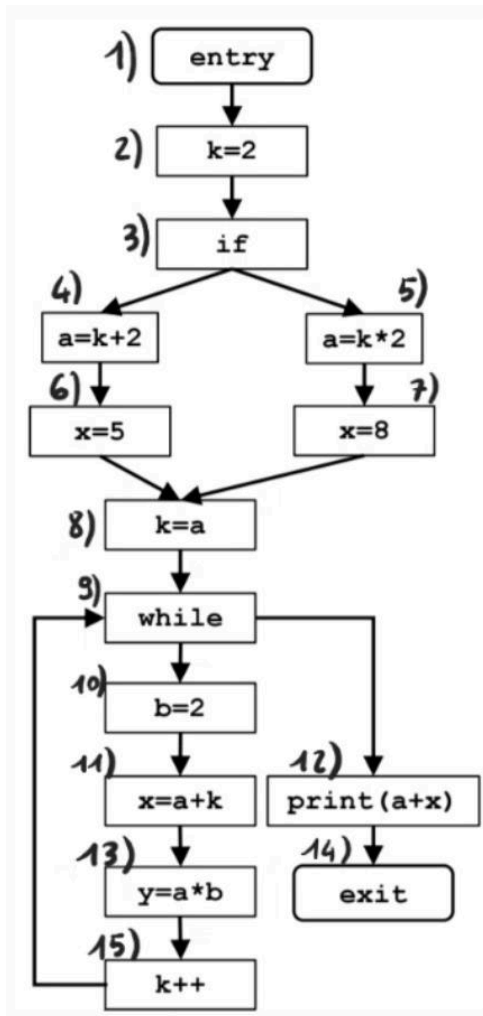
$$IN [BB2] = OUT[BB1] \cap OUT[BBn] = OUT[BB1] \text{ (Essendo BBn composto solo dalla variabile X).}$$

Nel caso in cui BBn avesse altre variabili il cui stato è diverso da -1, allora queste verrebbero comunque valutate nell'operazione di meet operation, escludendo quindi solamente le variabili che hanno come stato -1.

E' di particolare importanza il meet operator che si utilizza in per la gestione gli stati. Infatti deve poter confrontare fra di loro solamente le variabili che hanno un valore definito (o costanti o non costanti), ma non deve prendere in considerazione delle variabili mai definite (condizioni iniziali).

- costante (stato 1) ^ costante (stato 1) → il risultato dipende principalmente dall'intersezione fra le coppie <x,c1> e <x,c2> e darà valore costante (stato 1) solo se entrambi i valori c1 e c2 sono uguali, altrimenti fornisce stato 0.
- costante (stato 1) ^ non costante (stato 0) → non costante (stato 0).
Questo è anche il risultato del meet operatori applicato agli insiemi delle variabili.
- non costante (stato 0) ^ non costante (stato 0) → non costante (stato 0)
- costante (stato 1) ^ non valutato (stato -1) → costante (stato 1)
In questo modo l'initial condition non influenza il risultato della meet operation
- non costante (stato 0) ^ non valutato (stato -1) → non costante (stato 0)
- non valutato (stato -1) ^ non valutato (stato -1) → non valutato (stato -1)

ANALISI DELL'ESEMPIO



	ROUND 1		ROUND 2		ROUND 3	
	IN	OUT	IN	OUT	IN	OUT
1	/	{}	/	{}	/	{}
2	{}	{<k,2>}	{}	{<k,2>}	{}	{<k,2>}
3	{<k,2>}	{<k,2>}	{<k,2>}	{<k,2>}	{<k,2>}	{<k,2>}
4	{<k,2>}	{<k,2>, <a,4>}	{<k,2>}	{<k,2>, <a,4>}	{<k,2>}	{<k,2>, <a,4>}
5	{<k,2>}	{<k,2>, <a,4>}	{<k,2>}	{<k,2>, <a,4>}	{<k,2>}	{<k,2>, <a,4>}
6	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,5>}	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,5>}	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,5>}

7	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,8>}	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,8>}	{<k,2>, <a,4>}	{<k,2>, <a,4>, <x,8>}
8	{<k,2>, <a,4>}	{<k,4>, <a,4>}	{<k,2>, <a,4>}	{<k,4>, <a,4>}	{<k,2>, <a,4>}	{<k,4>, <a,4>}
9	{<k,4>, <a,4>}	{<k,4>, <a,4>}	{<a,4>}	{<a,4>}	{<a,4>}	{<a,4>}
10	{<k,4>, <a,4>}	{<k,4>, <a,4>, <b,2>}	{<a,4>}	{<a,4>, <b,2>}	{<a,4>}	{<a,4>, <b,2>}
11	{<k,4>, <a,4>, <b,2>}	{<k,4>, <a,4>, <b,2>, <x,8>}	{<a,4>, <b,2>}	{<a,4>, <b,2>}	{<a,4>, <b,2>}	{<a,4>, <b,2>}
12	{<k,4>, <a,4>}	{<k,4>, <a,4>}	{<a,4>}	{<a,4>}	{<a,4>}	{<a,4>}
13	{<k,4>, <a,4>, <b,2>, <x,8>}	{<k,4>, <a,4>, <b,2>, <x,8>, <y, 8>}	{<a,4>, <b,2>}	{<a,4>, <b,2>, <y, 8>}	{<a,4>, <b,2>}	{<a,4>, <b,2>, <y, 8>}
14	{<k,4>, <a,4>}	/	{<a,4>}	/	{<a,4>}	/
15	{<k,4>, <a,4>, <b,2>, <x,8>, <y, 8>}	{<k,5>, <a,4>, <b,2>, <x,8>, <y, 8>}	{<a,4>, <b,2>, <y, 8>}	{<a,4>, <b,2>, <y, 8>}	{<a,4>, <b,2>, <y, 8>}	{<a,4>, <b,2>, <y, 8>}